

01 — API Documentation

Per-endpoint reference for the FastAPI backend at `src/api.py`. For a flat one-line summary of every route see [04_ROUTES_AND_ENDPOINTS.md](#).

Conventions

- **Base URL** — `http://<host>:8000` (local dev: `http://localhost:8000`). In dev the Vite proxy at `frontend/vite.config.js` forwards every `/admin/*`, `/auth/*`, `/shifts/*`, `/machines/*`, `/query`, `/health`, `/workstation`, `/uploads/*` request to the backend.
- **Auth** — cookie-based.
 - `worker_session` — set by `POST /auth/worker-session` (12-hour expiry, `HttpOnly`, `SameSite=Lax`).
 - `stub_session` — set by `GET /auth/verify?token=...` after magic-link sign-in (30-day expiry).
 - Every `/admin/*` route is gated by the `require_admin` FastAPI dependency. Calls without a valid admin session return `401`.
- **Error envelope** — handlers raise `APIError(status, detail, code)`. The response body is `{"detail": "...", "code": "..."} for any non-2xx`.
- **Audit trail** — every state-changing admin call appends a row to `data/audit.jsonl` via `audit.append(...)`. Reads don't audit.

Public surface

GET /health

Liveness probe. Returns `{"status": "ok", "version": "0.1.0"}`. Used by Docker HEALTHCHECK and the nginx proxy.

POST /query

The main RAG endpoint. Worker sends a question, optional history, and `machine_filter`. Backend:

1. Embeds the question with `all-MiniLM-L6-v2`.
2. Retrieves top-K (5) chunks from ChromaDB filtered by `machine_filter`.
3. Calls Groq (Llama-3.1-70B) with the retrieved excerpts.
4. Computes `alert_score = severity × machine_significance`. Fires an alert email (deduped + snooze-aware) if `≥ ALERT_THRESHOLD`.
5. Appends to in-memory `_query_log` for analytics.

Body

```
{ "question": "...", "machine_filter": "INJECTION_MOLDING_MACHINE", "history": [{"role": "user", "content": "..."}]}
```

Returns — `{status, answer, sources, severity_level, alert_score, machine_significance, alert_fired}`.

GET /machines

List of indexed machines (`machine_id`, `display_name`, `category`, `significance`, `icon`, `description`, `suggested_questions`).
Public — workers see this on `/machines` to pick a chat target.

GET /machines/{machine_id}/parameters

Returns the admin-defined parameter spec for a machine: numeric readings + visual checks. Drives the dynamic form in `EndShiftModal`.

GET /machines/{machine_id}/shifts/recent?limit=5&phase=end

Most recent N shift logs for a machine. Used by:

- `HandoffBanner` (`limit=1&phase=end`) to find the previous shift's anomalies.
- `EndShiftModal` on pre-shift (`limit=3&phase=end`) to show recent readings inline.

POST /shifts/log (201 Created)

Worker submission. Backend computes anomalies + severity from the machine's parameter spec, freezes the result with the log, and inserts.

Body — `{machine_id, readings, visual_checks, notes, phase}`. `worker_label` is intentionally not accepted from the client; the backend derives it from `_derive_worker_label(payload, session, ip)` so it can't be spoofed.

POST /shifts/{log_id}/acknowledge

Worker-facing dismissal of the handoff banner. Sets `acknowledged = 1` on the row. Idempotent.

GET /workstation

Resolves the caller's IP to a bound machine via `data/workstations.json`. Returns `{bound: bool, machine: {...}}`. Used by `LandingPage` / `MachinesPage` to redirect bound workstations straight into their chat.

Authentication

POST /auth/request-link

Admin requests a magic sign-in link. The email must be in `ADMIN_EMAILS` (`.env`) — silently returns OK either way to avoid enumeration. Sends an email via Resend containing `<APP_BASE_URL>/auth/verify?token=...`. Token is single-use and expires in 15 min.

GET /auth/verify (redirect) and POST /auth/verify

Click target from the magic-link email. Sets the `stub_session` cookie and 302-redirects to `/`. Failure modes redirect to `/admin/login?login_error=expired|invalid`.

POST /auth/worker-session

Worker session creation. Body: `{domain}` where domain is one of `ALLOWED_DOMAINS`. Sets `worker_session` cookie (12h).

GET /auth/me

Returns the current session's role/domain/email. Admin session takes precedence over worker session when both cookies exist. `401` for unauthenticated.

POST /auth/logout

Clears both `worker_session` and `stub_session` cookies.

Admin — machines

All admin endpoints require `Depends(require_admin)`. They additionally append an audit-log row.

POST /admin/machines (202 Accepted)

Multipart form: `file` (PDF), `machine_id`, `display_name`, `description?`, `category?`, `significance?`, `icon?`. The PDF gets archived under `data/uploads/{machine_id}.pdf` and a background ingestion job (`_run_ingestion`) starts. Returns `{job_id, status: "queued"}`. Poll with `GET /admin/jobs/{job_id}`.

GET /admin/jobs/{job_id}

Job status: `queued` | `parsing` | `chunking` | `embedding` | `indexing` | `done` | `failed`. Plus `progress` (0.0–1.0), `step`, and `error` on failure.

GET /admin/machines

Same shape as public `/machines` but augmented with `uploaded_at`, `uploaded_by`, `pdf_size_bytes` for the admin dashboard.

PATCH /admin/machines/{machine_id}

Edit metadata in place — `display_name`, `description`, `category`, `significance`, `icon`, `suggested_questions`. Non-destructive: existing Chroma chunks, parameters, and shift logs are untouched. Use this instead of delete + re-create for renames.

DELETE /admin/machines/{machine_id}

Hard delete: drops all Chroma chunks tagged `machine == machine_id`, removes the archived PDF, drops the metadata row. Does NOT cascade-delete shift logs (those stay for audit).

POST /admin/machines/{machine_id}/reingest (202)

Re-runs `_run_ingestion` against the existing `data/uploads/{machine_id}.pdf`. Drops old chunks first to avoid duplicates. Parameters and shift logs survive.

PUT /admin/machines/{machine_id}/parameters

Replaces the parameter spec (numeric readings + visual checks). Body: `{numeric_readings: [...], visual_checks: [...]}`. Existing shift logs are not retroactively reclassified — they keep the anomalies that were computed at submit time.

Admin — shift logs

GET /admin/shifts?machine_id=&limit=100&phase=

List shift logs, newest first. Optional filters: `machine_id`, `phase` (`start` | `end`).

POST /admin/shifts/{log_id}/acknowledge

Admin-side acknowledge (separate from worker `/shifts/{id}/acknowledge` for attribution). Idempotent.

POST /admin/shifts/{log_id}/void

Soft-delete a shift log. Body: `{reason}` (required, 1–500 chars). Row stays in the table for audit but `void_at` is set; `latest_shift_log()` filters voided rows so the handoff banner skips them.

Admin — alerts

GET /admin/alerts

`{alerts: [...], threshold, snoozes: {machine_id: iso_until}, dedup_seconds}`. Newest first.

DELETE /admin/alerts

Clears the in-memory `_alerts` list. Returns `{ok, cleared}`.

POST /admin/alerts/{alert_id}/acknowledge

Sets `acknowledged_at` + `acknowledged_by` on the alert. Doesn't remove it from history.

POST /admin/alerts/snooze

Body: `{machine_id, minutes}`. `minutes = 0` lifts the snooze. Bounded 0–1440. While a machine is snoozed, alerts that would have fired are suppressed (and logged to stdout) but `_query_log` still records the underlying query.

POST /admin/alerts/test (201)

Injects a synthetic alert end-to-end (including the email if `ADMIN_EMAILS` is configured) so ops can verify the Resend pipeline.

Admin — config (runtime tunable)

GET /admin/config

Returns the current effective values:

- `alert_threshold` (1–25)
- `alert_dedup_seconds` (0–86400)
- `admin_emails` (read-only, from `.env`)

- `allowed_domains` (read-only)

PATCH /admin/config

Updates the tunable fields and persists to the `app_config` SQLite table. `_load_runtime_config()` re-reads after every write so the change is live without a restart.

Admin — analytics

GET /admin/analytics

Aggregates `_query_log` into the dashboard widgets. Every filter composes (AND):

Param	Notes
<code>days</code>	Last N days. Wins over <code>date_from / date_to</code> if both present.
<code>date_from / date_to</code>	Inclusive YYYY-MM-DD window.
<code>category</code>	Machine category (<code>Manufacturing</code> , <code>Fabrication</code> , ...).
<code>severity</code>	Exact match 1..5.
<code>shift</code>	<code>Morning</code> / <code>Afternoon</code> / <code>Night</code> , classified from <code>asked_at</code> via <code>_get_shift</code> .
<code>machine_id</code>	Narrows everything including KPIs + severity donut.

Returns `totals` , `per_machine` , `code_frequency` , `severity_distribution` , `queries_per_hour_24h` , `top_questions` , `failure_likelihood` , `depreciation` . The hourly bars + failure-likelihood + depreciation intentionally ignore filters — they're time-bound or per-asset, not per-query.

POST /admin/_seed-analytics?count=80&replace=false

Demo-only — injects synthetic `_query_log` entries so the dashboard renders something out of the box. **Not for production.**

GET /admin/audit?limit=200&action_prefix=

Most recent audit-log entries from `data/audit.jsonl` . Filterable by action prefix (`auth.` , `machine.` , `shift.` , `alert.` , `config.`).

Static assets

GET /uploads/icons/{filename}

Mounted with FastAPI `StaticFiles` at `data/uploads/icons/` . Serves admin-uploaded custom machine icons. PDFs in `data/uploads/` are NOT exposed.